Exploring Optimisation Paradigms for Mixed Discrete–Continuous LoRA Hyperparameter Spaces in Transformer Fine-Tuning

***Abstract*—This work investigates the use of metaheuristic optimisation for tuning Low-Rank Adaptation (LoRA) hyperparameters when fine-tuning DistilBERT on the dair-ai/emotion classification dataset under strict GPU constraints. LoRA introduces a mixed discrete–continuous search space and requires expensive model evaluations, making traditional optimisation methods ineffective. To address this, four optimisation paradigms are examined within a unified framework: Random Search, the population-based Aquila Optimizer, a hybrid discrete PSO with genetic operators (DPSO–GA), and the trajectory-based Blindfolded Spiderman Optimisation (BSO). Results show that BSO achieves the highest validation accuracy (94%), the strongest mean and median performance, and the fastest convergence while requiring substantially fewer evaluations than the population-based methods. A multi-objective extension using MOEA/D-DE further reveals trade-offs between accuracy and parameter efficiency, producing Pareto-optimal configurations up to 90.2% accuracy. Overall, the study demonstrates that lightweight trajectory-based metaheuristics are highly effective for LoRA hyperparameter optimisation in resource-limited settings.**

***Index Terms*—LoRA, hyperparameter optimisation, metaheuristics, DistilBERT, emotion classification**

I. Introduction and Literature Review

A. Introdctution

Transformer-based language models have reshaped modern natural language processing, powering applications that range from sentiment analysis to conversational assistants. Models such as BERT and DistilBERT achieve this by learning deep contextual representations; however, their impressive performance comes at a cost: fully fine-tuning them requires updating millions of parameters, consuming significant GPU time and memory. For many practitioners, particularly those working with limited computational resources, this makes traditional fine-tuning impractical. Parameter-efficient fine-tuning (PEFT) methods were introduced to address this challenge, with Low-Rank Adaptation (LoRA) quickly becoming one of the most widely adopted techniques. LoRA modifies only a small subset of the model using low-rank matrix updates, allowing efficient adaptation without sacrificing accuracy [1]. Although LoRA dramatically reduces computational overhead, it introduces a new difficulty: its performance is highly sensitive to several interacting hyperparameters. Choices such as the learning rate, warm-up ratio, dropout, LoRA rank r, scaling factor α, and even which internal modules to adapt can each influence the stability and final accuracy of fine-tuning. Recent studies on PEFT methods highlight that the effectiveness of LoRA varies significantly across tasks, and seemingly small adjustments can lead to large differences in performance down the line [2]. This makes hyperparameter optimization essential, yet far from straightforward.

Optimizing LoRA hyperparameters is challenging for several reasons. Each evaluation requires fine-tuning the underlying model for several epochs, making the search process computationally expensive. The search space itself is also complex: some parameters are continuous, others are discrete, and their interactions are highly non-linear. Moreover, validation accuracy after fine-tuning is a black-box quantity with no analytical gradient. These characteristics render traditional optimization strategies such as grid search and random search inefficient, and they prevent the use of gradient-based methods. As a result, researchers have increasingly turned to metaheuristic optimization methods, which are designed to explore large, irregular, and expensive search spaces.

Metaheuristics provide a flexible framework for navigating such problems. Population-based algorithms, such as the Aquila Optimizer (AO), employ diverse exploration strategies that help escape poor regions of the search space and identify promising areas efficiently [1]. Hybrid evolutionary methods, such as PSO augmented with genetic operators, combine rapid convergence with improved diversity maintenance, making them effective in mixed discrete–continuous settings like LoRA tuning [3]. In contrast, trajectory-based single-solution algorithms such as Blindfolded Spiderman Optimization (BSO) embrace a more lightweight, step-by-step refinement process, which is particularly advantageous when each evaluation is costly [4]. When multiple objectives must be considered simultaneously, such as maximizing accuracy while minimizing trainable parameters, multi-objective evolutionary algorithms like MOEA/D-DE offer principled ways to approximate efficient trade-offs through decomposition and differential evolution operators [5].

B. Literature Review

Low-Rank Adaptation (LoRA) introduces low-rank update matrices into selected transformer layers, allowing efficient fine-tuning while keeping most parameters frozen. However, the effectiveness of LoRA depends strongly on several interacting hyperparameters, including the rank r, scaling factor α, dropout, and the choice of target modules. The original LoRA paper highlights that these settings significantly affect how much task-specific capacity is added to the frozen backbone [6]. Broader surveys on

parameter-efficient fine-tuning likewise emphasize that PEFT methods remain highly sensitive to configuration choices and that optimal hyperparameters vary substantially across tasks and architectures [2]. This sensitivity makes LoRA hyperparameter optimization essential for achieving stable and high-performing models. Optimizing LoRA hyperparameters is challenging because the search space contains a mixture of continuous variables (e.g., learning rate, dropout), discrete structural parameters (rank r, scaling α), and categorical design choices such as target modules. These dimensions interact in complex ways that make the relationship between hyperparameters and model performance highly irregular. Since each configuration must be evaluated through an actual fine-tuning run, the optimization landscape is expensive to sample and inherently noisy. Recent studies on automated LoRA tuning highlight that its hyperparameter space is difficult to navigate efficiently and that performance can shift abruptly in response to seemingly minor parameter adjustments. Broader analyses of hyperparameter optimization in deep learning similarly note that such mixed and non-differentiable landscapes pose significant challenges for traditional search strategies, particularly when evaluation costs are high [7]. As a result, methods reliant on smoothness, gradients, or exhaustive enumeration are poorly suited to this problem.

Metaheuristic algorithms are well-suited for this class of optimization problems because they do not require gradient information and can navigate high-dimensional, irregular, and noisy search spaces. A recent comprehensive review of metaheuristic-based hyperparameter optimization for deep neural networks reports that such methods often outperform naive search strategies when objective evaluations are expensive or non-convex [7]. Metaheuristics offer diverse search mechanisms: population-based strategies emphasize exploration, hybrid evolutionary algorithms balance convergence and diversity, and single-point trajectory-based methods refine solutions efficiently under limited evaluation budgets. These characteristics align well with the demands of LoRA hyperparameter optimization. The four optimization paradigms chosen in this project represent distinct approaches to black-box search. The Aquila Optimizer (AO) provides a population-based method with multi-phase exploration mechanisms that are effective in complex global optimization landscapes [6]. The PSO–GA hybrid combines the fast, gradient-free convergence of particle swarm optimization with genetic operators that preserve diversity and enable discrete adaptations, making it suitable for mixed search spaces such as LoRA's hyperparameter domain [3]. Blindfolded Spiderman Optimization (BSO) is a trajectory-based single-solution algorithm designed for efficient search under tight evaluation budgets; its directional and corrective movements support fine-grained local improvement without large population overheads [4]. Finally, MOEA/D-DE introduces a multi-objective perspective by decomposing the accuracy–complexity trade-off into subproblems and solving them using differential-evolution operators, enabling systematic exploration of Pareto-optimal solutions in bi-objective LoRA tuning [5].

Although LoRA has become one of the most widely used PEFT techniques, prior research has focused mainly on architectural variants or heuristic hyperparameter choices rather than systematic optimization. Recent studies employing Bayesian optimization, black-box optimization, or meta-learning address parts of the problem, but none provide a unified comparison across fundamentally different metaheuristic paradigms. In particular, no existing work jointly evaluates population-based, hybrid evolutionary, trajectory-based, and multi-objective algorithms under a shared evaluation framework on the same LoRA search space. This leaves an open question regarding which optimization class is most effective for LoRA tuning under realistic compute constraints. LoRA hyperparameter optimization presents a demanding mixed-variable search problem characterized by non-linear interactions and costly evaluations. Metaheuristic algorithms offer an attractive solution because they can operate effectively in irregular, gradient-free settings and support diverse search strategies. The four methods considered in this study, AO, PSO–GA, BSO, and MOEA/D-DE, capture complementary optimization philosophies, providing a broad basis for comparing how different algorithmic classes perform on LoRA tuning. The following subsection summarises the specific contributions of this work.

C. Problem Statement and Objectives

Fine-tuning DistilBERT with Low-Rank Adaptation (LoRA) requires choosing several coupled hyperparameters (e.g., learning rate, warmup ratio, LoRA rank, scaling factor, dropout, and target modules). These form a high-dimensional mixed search space (continuous, discrete, and categorical), where each evaluation is expensive because it involves building a new LoRA configuration and running multi-epoch GPU training.

How can we efficiently search the LoRA hyperparameter space for DistilBERT, under a strict GPU budget, to obtain high validation accuracy and stable performance across random seeds?

To address this, we define the following objectives:

- Efficiently explore the mixed LoRA hyperparameter space under a fixed evaluation budget (limited number of trials and epochs).
- Compare different optimization paradigms (baseline random search, population-based AO, hybrid DPSO–GA, trajectory-based BSO, and multi-objective MOEA/D-DE) on the same DistilBERT+LoRA task.
- Quantify the trade-offs between validation accuracy, convergence behaviour, robustness across seeds, and computational cost (GPU time and parameter efficiency).
- Identify which optimization strategy offers the best overall balance for practical LoRA fine-tuning of DistilBERT.

D. Contributions

The contributions of this work are as follows: We develop a unified framework for evaluating LoRA-based fine-tuning of DistilBERT under a fixed training budget. We adapted 3 metaheuristic algorithms, AO, PSO-GA–GA and BSO to the mixed discrete–continuous LoRA hyperparameter space. We formulate a bi-objective accuracy–complexity problem and apply a multi-objective evolutionary algorithm to analyze the resulting trade-offs. Despite LoRA's increasing use, there is still limited research that directly compares a broad range of optimization strategies for tuning its hyperparameters. This motivates a systematic evaluation of different metaheuristic families to identify which approaches achieve the best balance between accuracy and computational cost under strict resource constraints

II. Optimisation Algorithms

We employ several metaheuristic optimisation algorithms to search the high-dimensional LoRA hyperparameter space under strict evaluation budgets. The Aquila Optimizer (AO) serves as our population-based method and is described in detail in Appendix A-B, drawing on the hunting-inspired operators introduced in [8]. The following subsections outline the remaining algorithms used in this study.

A. Algorithm 1: Hybrid Discrete PSO with Genetic Algorithm

In our chosen Algorithm 1, we optimise the LoRA hyperparameters of a DistilBERT classifier using a *Hybrid Discrete Particle Swarm Optimisation with Genetic Algorithm operators* (DPSO-GA). Our implementation follows the hybrid scheme proposed by Fan and Liang for directional wireless sensor networks [3] and adapts it to the continuous-discrete hyperparameter space of LoRA fine-tuning.

1) Justification for the Hybrid DPSO-GA Approach: Pure Genetic Algorithms (GA) are strong global explorers, but they often converge slowly and may disrupt good partial solutions through crossover and mutation. Standard Particle Swarm Optimization (PSO) converges quickly by exploiting the current global best, but it can easily get trapped in local optima, especially in highly non-convex deep learning hyperparameter spaces.

Our LoRA hyperparameter search has a mixed search space with continuous (learning rate, dropout), discrete (rank r, α), and categorical (target modules) dimensions. DPSO is well-suited to this setting because we encode all hyperparameters as a continuous vector in $[0,1]^6$, then map each dimension to continuous, discrete, or categorical values via simple transforms. This lets PSO operate in a smooth bounded space while still respecting discrete and categorical choices. A hybrid DPSO-GA is therefore attractive because:

I. **DPSO phase:** particles are driven towards their own best configuration and the global best configuration, accelerating convergence to promising regions.

II. **GA phase:** tournament selection, uniform crossover, and mutation inject new candidate configurations that can escape local optima and maintain diversity in the population.

III. **Shared population:** both DPSO and GA act on the same swarm each iteration, so PSO exploits strong individuals created by GA, and GA uses PSO-refined particles as parents, reusing information efficiently.

2) Justification of Algorithm Hyperparameters: Table IV lists the hyperparameters used in our implementation of the hybrid DPSO-GA optimiser.

I. Population size $N = 5$ and iterations $T = 10$: Each seed evaluates 50 configurations ($N \times T$). A small swarm limits the number of expensive fine-tuning runs while still allowing some diversity in the mixed continuous/discrete/categorical space.

II. DPSO parameters: $\omega = 0.1$ adds occasional random perturbations without wiping out useful structure. $c_1 = c_2 = 0.5$ gives a simple 50/50 balance between following each particle's own best solution and the swarm's global best.

III. GA parameters ($p_{\text{cx}} = 0.8$, $p_{\text{mut}} = 0.1$): A high crossover rate (0.8) encourages recombination of good solutions; a moderate mutation rate (0.1) injects randomness without degenerating into random search.

IV. Tournament size: $k = 3$ provides moderate selection pressure: better particles are more likely to become parents, but weaker ones still occasionally contribute, helping to maintain diversity in a small population.

3) Hybrid DPSO-GA Dynamics: **(a) DPSO phase (exploitation):** At each iteration t, every particle i starts from its current position $\mathbf{x}_i^t \in [0,1]^6$, which encodes a LoRA hyperparameter configuration. The DPSO phase updates this position in three simple steps:

i. **Mutation F_1 (random perturbation).** For each hyperparameter dimension, with probability ω we replace the current value by a new random value in $[0,1]$; otherwise, we keep the old value. This allows the particle to jump to new regions of the search space occasionally.

ii. **Cognitive crossover F_2 (move towards personal best).** Each particle keeps track of its own best position so far (*personal best*). For each dimension, with probability c_1 we copy the value from this personal-best position; otherwise, we keep the mutated value from F_1.This step encourages each particle to move back towards configurations that previously gave that same particle a high validation accuracy.

iii. **Social crossover F_3 (move towards global best).** The swarm also tracks the best position found by any particle so far (the *global best*). For each dimension, with probability c_2, we copy the value from the global-best position; otherwise, we keep the value from the previous step (F_2). This encourages all particles to move towards the best solution discovered by the group.

After applying F_1, F_2 and F_3, the new position is clipped to $[0,1]^6$, decoded into a LoRA configuration, and evaluated by fine-tuning DistilBERT+LoRA for 3 epochs. The resulting validation accuracy is the fitness value. If this fitness is better than the particle's previous *personal best*, we update its personal best position and fitness. If it is also better than the swarm's global best, we update the global-best position and fitness as well.

(b) GA phase (exploration): After the DPSO phase has

updated all particles, we run a GA phase to add diversity and explore new configurations:

i. **Tournament selection (choose parents).** To select a parent, we randomly sample a small group of particles (we use $k = 3$) and pick the one with the highest fitness. Repeating this procedure twice gives two parents i and j.

ii. **Uniform crossover (recombine parents).** With probability p_{cx} we create an offspring by mixing the two parents: for each hyperparameter, we randomly choose the value from parent i or parent j with equal probability. If crossover does not occur, we simply copy one of the parents as the offspring.

iii. **Mutation (random perturbation).** For each dimension of the offspring, with probability p_{mut} we replace the current value by a new random value in $[0, 1]$; otherwise we keep it unchanged. This step allows the GA phase to discover hyperparameter values that are not present in the current swarm.

iv. **Decode, evaluate, and elitist replacement.** The offspring position is clipped to $[0, 1]^6$, decoded into a LoRA configuration, and evaluated by training DistilBERT+LoRA for 3 epochs to obtain its validation accuracy. We then apply *elitist replacement* : if the offspring has a higher fitness than one of the current particles (e.g. the worst-performing one), we replace that particle with the new offspring. This ensures that strong candidates are preserved while weaker ones are gradually removed.

4) Pseudocode: Algorithm 1 summarises the overall procedure.

Algorithm 1 Hybrid Discrete PSO with GA operators (DPSO_GA) for LoRA Tuning

Initialise parameters: population size P, max iterations $t_{\max}$, probabilities $\omega, c_1, c_2, p_{\mathrm{cx}}, p_{\mathrm{mut}}$.
For each particle, randomly initialize its position in $[0, 1]^6$.
For each particle, decode its position to LoRA hyperparameters, train for 3 epochs, compute validation accuracy, and set *particle_best* (its personal-best position) and *particle_best_fitness* (its personal-best fitness) to these values.
Set *global_best* to the particle with the highest fitness.
Set $t \leftarrow 0$.
while $t < t_{\max}$ **do**
  DPSO phase (exploit best solutions)
  for each particle **do**
    Apply mutation F_1: randomly change some dimensions with prob. ω).
    Apply cognitive crossover F_2: copy genes from this particle's personal-best position (*particle_best*) with prob. c_1.
    Apply social crossover F_3: copy genes from the swarm's global-best position (*global_best*) with prob. c_2.
    Clip position to $[0, 1]^6$, decode, train for 3 epochs and compute fitness.
    Update *particle_best* and *global_best* if fitness improves.
  end for
  GA phase (explore new solutions)
  Build a mating pool by selecting fitter particles more often.
  Create offspring by uniform crossover (prob. p_{cx}) and mutation (prob. p_{mut}), then decode, train for 3 epochs and evaluate their fitness.
  Combine current particles and offspring; keep the P particles with highest fitness as the new swarm and update *global_best* if needed.
  $t \leftarrow t + 1$.
end while
return *global_best* (best LoRA hyperparameters) and *global_best_fitness* (best validation accuracy).

B. Algorithm 2: Blindfolded Spiderman Optimization (BSO)

1) Motivation and Justification: Blindfolded Spiderman Optimization (BSO) is a single-point metaheuristic that follows a trajectory through the search space rather than evolving a population of candidate solutions. It was proposed as a successor to the Buggy Pinball optimizer, retaining the intuition of a "moving point" that explores the landscape while simplifying the update rules and making the method applicable to both continuous and discrete problems [4]. In contrast to population-based methods such as PSO or GA, which distribute the evaluation budget across many individuals, BSO concentrates all evaluations on one incumbent solution at a time and only continues a trajectory while it is making progress.

This behaviour is particularly attractive for LoRA hyperparameter optimisation. In our setting, each fitness evaluation involves building a LoRA-adapted DistilBERT model, fine-tuning it for three epochs on the training split, and computing the validation accuracy. This makes evaluations expensive and limits the number of configurations that can be tested. Using a large population would quickly exhaust the evaluation budget. BSO instead focuses on refining a single candidate along a piecewise linear path, terminating trajectories early when no further improvement is found and restarting from a new random point. This design offers a good compromise between exploration and exploitation under a small evaluation budget. Furthermore, BSO operates naturally in a continuous search space, while our implementation uses a decoding function to handle the discrete and categorical choices of the LoRA hyperparameters. These properties make BSO a reasonable algorithmic choice for this problem and complementary to the population-based and hybrid methods considered in this study.

In our implementation, each candidate solution is represented by a vector

$$x \in [0, 1]^6.$$

The six components of x are mapped through a `decode` function to the actual LoRA configuration: learning rate (via log scaling), warm-up ratio, LoRA rank r, scaling factor α,

dropout and target-module selection (e.g. attention-only versus attention+FFN). The fitness function $f(x)$ is the validation accuracy obtained after running the standard three-epoch fine-tuning procedure with this configuration. The goal of BSO is to maximise $f(x)$ given a fixed number of evaluations.

2) Trajectory-Based Search Dynamics: BSO explores the search space through a sequence of trajectories. At the beginning of the algorithm, a global best solution $(x_{\text{best}}, f_{\text{best}})$ is initialised by sampling a random point $x_{\text{best}} \in [0,1]^6$, decoding it into LoRA hyperparameters and evaluating its fitness. The algorithm then performs T_{traj} trajectories. At the start of each trajectory, a fresh point x is drawn at random from $[0,1]^6$, evaluated to obtain f_x, and compared against the current global best. A step-size parameter `step_size` is initialised, and a pair of angular parameters (θ, ϕ) that define an initial search direction are sampled.

Within each trajectory, BSO iteratively attempts to improve the current solution x using two types of moves:

a) Forward move (y-step): Given the current point x and angles (θ, ϕ), a direction vector d is constructed (for example, by mapping (θ, ϕ) to a unit vector in $\mathbb{R}^6$ and adding small random noise), and a candidate

$$y = \text{clip}\big(x + \text{step_size} \cdot d\big) \tag{1}$$

is proposed, where clip enforces that each coordinate remains within $[0,1]$. This candidate is decoded into a LoRA configuration and evaluated to obtain $f_y = f(y)$. If the new configuration improves the fitness ($f_y > f_x$), the move is accepted: the state is updated to $x \leftarrow y$, $f_x \leftarrow f_y$, and, if $f_x > f_{\text{best}}$, the global best pair $(x_{\text{best}}, f_{\text{best}})$ is also updated. To encourage a gradual shift from coarse exploration to fine local search, the step size is multiplied by a factor $0 < \rho < 1$ (e.g. $\rho = 0.85$), and the angles (θ, ϕ) are perturbed slightly to adjust the direction without completely randomizing it.

b) Corrective move (u-step): If the y-step does not improve the solution ($f_y \leq f_x$), BSO performs a corrective move. The angles (θ, ϕ) are perturbed more strongly to generate a new direction d_u, and a smaller step is attempted:

$$y_u = \text{clip}\big(x + \gamma \cdot \text{step_size} \cdot d_u\big), \tag{2}$$

where $\gamma \in (0,1)$ is a constant (e.g. 0.3) that scales the corrective move. As before, y_u is decoded and evaluated to obtain f_u. If $f_u > f_x$, the move is accepted, the current solution is updated to $x \leftarrow y_u$, and the global best is refreshed where appropriate. If neither the y-step nor the u-step produces an improvement, the current trajectory is terminated prematurely, and the algorithm proceeds to the next trajectory.

This strategy ensures that trajectories are only extended when they yield genuine progress. When a region of the search space appears locally unproductive, BSO abandons it and reallocates the evaluation budget to a new random starting point. Because the method works directly in the continuous $[0,1]^6$ box, the mixed discrete–continuous nature of the LoRA search space is fully encapsulated inside the `decode` function, keeping the core search logic simple.

3) Pseudocode: Algorithm 2 summarises the BSO procedure used in this work. The notation is consistent with the other optimisation algorithms: `decode` maps a point in $[0,1]^6$ to a LoRA configuration, and `eval` returns validation accuracy after three-epoch fine-tuning.

III. Experimental Results

This section describes the experimental setup and presents the results of the single-objective optimisation algorithms.

A. Dataset and Preprocessing

We use the `dair-ai/emotion` [9] dataset, which consists of short English text snippets labelled with six emotions: *anger*, *disgust*, *fear*, *joy*, *sadness*, and *surprise*. The original train/validation/test splits are retained, but the training set is subsampled to 3000 examples to respect the GPU budget. Text is toktokenized the `distilbert-base-uncased` [10] tokenizer using a maximum sequence length of 128, applying truncation and padding as required. Preprocessing is implemented with the Hugging Face `datasets` library, producing input IDs, attention masks, and labels. No additional text cleaning or normalization is applied so that the setup remains comparable to standard Emotion benchmarks.

B. Model and Training Configuration

Each trial evaluates a DistilBERT classifier fine-tuned with LoRA adapters. Adapters are inserted into either **attention-only** modules {`q_lin`, `v_lin`} or **attention + feedforward** modules {`q_lin`, `v_lin`, `ffn.lin1`, `ffn.lin2`}, as determined by the candidate configuration. The transformer backbone is frozen; only the LoRA weights and the classification head are updated.

All optimisation algorithms are run with four random seeds (1, 42, 999, 1234), with 10–20 trials per seed depending on the evaluation budget. Each trial fine-tunes the model for three epochs using cross-entropy loss, the AdamW optimiser, a linear warm-up schedule, and gradient clipping with max-norm 1.0. The warm-up steps are derived from the candidate warm-up ratio. Batch size (32), classifier dropout, and weight decay are fixed across all experiments. The fitness value used by all algorithms is the validation accuracy after the third epoch. For every trial, we also log wall-clock training time and peak GPU memory usage to enable a fair comparison of computational cost.

C. Hyperparameter Search Space

The optimisation algorithms operate over a mixed discrete/continuous search space that covers LoRA parameters and learning-rate scheduling. Each algorithm searches in the normalised space $[0,1]^6$ and uses the same decoding scheme to map continuous vectors to concrete hyperparameters. The ranges are summarised in Table I.

Algorithm 2 Blindfolded Spiderman Optimization (BSO) for LoRA HPO

Require: T_{traj} (number of trajectories), $S_{\max}$ (max. steps per trajectory), step_init (initial step size), `decode`$(\cdot)$, `eval`$(\cdot)$

Ensure: $x_{\text{best}}, f_{\text{best}}$

$x_{\text{best}} \leftarrow$ random point in $[0,1]^6$
$f_{\text{best}} \leftarrow$ `eval`(`decode`(x_{best}))
for $t = 1$ **to** T_{traj} **do**
  $x \leftarrow$ random point in $[0,1]^6$
  $f_x \leftarrow$ `eval`(`decode`(x))
  if $f_x > f_{\text{best}}$ **then**
    $x_{\text{best}} \leftarrow x$; $f_{\text{best}} \leftarrow f_x$
  end if
  step_size $\leftarrow$ step_init
  initialise angles (θ, ϕ) at random
  for $s = 1$ **to** $S_{\max}$ **do**
    $d \leftarrow$ `direction_vector`(θ, ϕ) {unit direction in $\mathbb{R}^6$}
    $y \leftarrow \text{clip}(x + \text{step_size} \cdot d)$
    $f_y \leftarrow$ `eval`(`decode`(y))
    if $f_y > f_x$ **then**
      {successful y-step}
      $x \leftarrow y$; $f_x \leftarrow f_y$
      if $f_x > f_{\text{best}}$ **then**
        $x_{\text{best}} \leftarrow x$; $f_{\text{best}} \leftarrow f_x$
      end if
      step_size $\leftarrow \rho \cdot$ step_size $\{0 < \rho < 1\}$
      $(\theta, \phi) \leftarrow$ `small_perturb`(θ, ϕ)
    else
      {attempt u-step}
      $(\theta, \phi) \leftarrow$ `larger_perturb`(θ, ϕ)
      $d_u \leftarrow$ `direction_vector`(θ, ϕ)
      $y_u \leftarrow \text{clip}(x + \gamma \cdot \text{step_size} \cdot d_u)$
      $f_u \leftarrow$ `eval`(`decode`(y_u))
      if $f_u > f_x$ **then**
        {successful u-step}
        $x \leftarrow y_u$; $f_x \leftarrow f_u$
        if $f_x > f_{\text{best}}$ **then**
          $x_{\text{best}} \leftarrow x$; $f_{\text{best}} \leftarrow f_x$
        end if
      else
        {no improvement: end trajectory}
        break
      end if
    end if
  end for
end for
return $x_{\text{best}}, f_{\text{best}}$

TABLE I
HYPERPARAMETER SEARCH SPACE FOR LORA TUNING AND OPTIMISATION.

Hyperparameter	Range / Values
Learning rate η	Log-uniform in $[10^{-6}, 5 \times 10^{-4}]$
Warm-up ratio ω	Continuous in $[0.0, 0.2]$
LoRA rank r	$\{2, 4, 8, 16, 32\}$
LoRA alpha α	$\{8, 16, 32, 64, 128\}$
LoRA dropout p_{drop}	Continuous in $[0.0, 0.3]$
Target modules m_{target}	{attention-only, attention+FFN}

D. Resource Monitoring and Logging

To compare computational efficiency, all four algorithms share a common logging and monitoring framework. Before each evaluation, CUDA peak-memory statistics are reset; after training, the maximum allocated memory is recorded along with validation accuracy and training time. Models are deleted and GPU cache is cleared between trials. All measurements are stored in a unified JSON format (Appendix 1), which is used to derive convergence curves, accuracy distributions, and cost statistics.

E. Results and Discussion

We benchmark Random Search, Aquila, PSO+GA, and Blindfolded Spiderman Optimization (BSO) in terms of accuracy, convergence behaviour, and computational cost. Table II summarises the quantitative results across seeds and across all fitness evaluations.

1) Quantitative Model Comparison: Table II shows that BSO achieves the strongest overall performance. It attains the highest best accuracy (94%), mean accuracy (93.91%), and median accuracy (93.11%), indicating that both its best and typical solutions are superior to the baselines. Its worst accuracy (33.50%) is substantially better than Random Search (11.05%) and comparable to PSO+GA (35.20%), suggesting that BSO rarely samples extremely poor configurations. We report worst and median accuracies to characterize performance across the entire optimization run, rather than only at the best point. Worst accuracy also highlights how poorly the method can perform when it samples a weak configuration, while, on the other hand, median accuracy shows the overall quality of the solutions an algorithm is generating most of the time. BSO also reaches these results with only 102 evaluations, compared with 400 for Aquila and 420 for PSO+GA, making it the most evaluation-efficient method. PSO+GA yields competitive best and median values but at a higher cost, while Aquila is stable yet slightly weaker. Random Search performs worst across all metrics, confirming that naive sampling is ineffective in this hyperparameter space.

TABLE II
MODEL COMPARISON ACROSS SEEDS AND ACROSS ALL EVALUATIONS. ALL ACCURACIES ARE REPORTED IN PERCENTAGE (%).

Measures	Random Search	Aquila	PSO + GA	Blindfolded Spiderman
Across 4 Seeds				
Best Accuracy(%)	89.40	91.20	91.65	94.00
Mean(%) ± Std(%)	85.16 ± 5.15	91.11 ± 0.07	90.98 ± 0.60	93.91 ± 0.07
Across All Evaluations				
Total Trials/ Trajectories	20	20	10	10
Total Evaluations	80	400	420	102
Worst Accuracy(%)	11.05	4.05	35.20	33.50
Median Accuracy(%)	45.78	79.90	89.40	93.11

2) Convergence and Evaluation Efficiency: Figure 1 illustrates convergence in terms of the best validation accuracy per iteration or trajectory. BSO rapidly reaches above 94% within its first two trajectories and then remains stable, indicating fast and reliable convergence. Aquila also improves quickly but saturates around 90%, while PSO+GA shows a slower, more gradual increase before plateauing below BSO. Random Search exhibits irregular progress and plateaus early at substantially lower accuracy.

Figure 2 compares validation accuracy against the number of objective evaluations. BSO achieves 93–94% accuracy in fewer than 20 evaluations, whereas PSO+GA and Aquila require many more evaluations (60–80+) to approach their respective plateaus. The confidence bands highlight that BSO is also the most consistent across seeds, while Random Search and PSO+GA display much larger variability. Overall, these plots confirm that BSO provides the best trade-off between convergence speed and final accuracy.

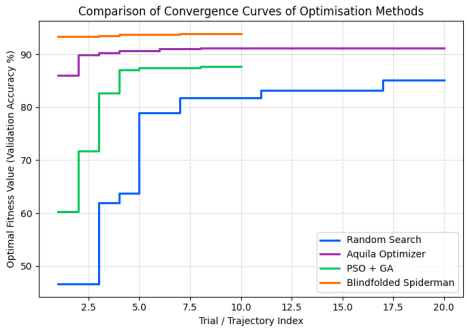

Fig. 1. Convergence curves: best validation accuracy per iteration/trajectory.

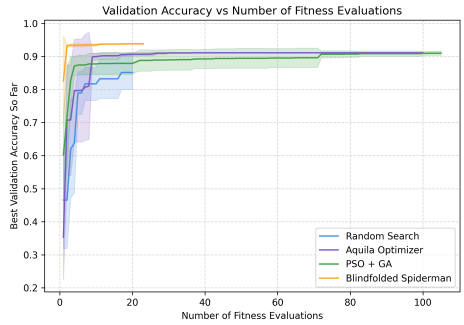

Fig. 2. Validation accuracy versus number of objective evaluations per seed.

3) Computational Cost and Resource Usage: Figure 3 reports average GPU memory usage per evaluation. PSO+GA has the lowest footprint (0.94 GB) indicating that although it evaluates many candidates, each evaluation requires relatively lightweight model instantiation, followed closely by BSO (1.11 GB) which is notable given that it achieves the best accuracy and fastest convergence; this confirms that its performance advantage does not rely on high memory consumption. Aquila Optimizer requires moderately higher memory (1.58 GB), reflecting its repeated reconstruction of LoRA configurations and slightly larger model footprints during evaluation. In contrast, Random Search records the highest peak memory usage (1.77 GB). Although Random Search only uses the GPU for short bursts, since, as seen in figure 4, each trial is independent, and no iterative population is maintained, its per-evaluation GPU demand spikes to the highest levels among all methods leading to the fact that Random Search has the worst memory efficiency.

The average training time per seed in Figure 4 shows that Random Search is the cheapest (approximately 1504 s) but also the least effective. Aquila and BSO incur higher costs (7503.9 s and 8129.6 s respectively) due to structured iterative search, yet they achieve substantially better accuracies. PSO+GA is the most computationally expensive (8899 s per seed) because it evaluates many candidates per generation. Taken together, the results indicate that BSO offers the best balance between accuracy and cost: it is not the cheapest method, but it delivers the highest performance with a moderate training-time overhead.

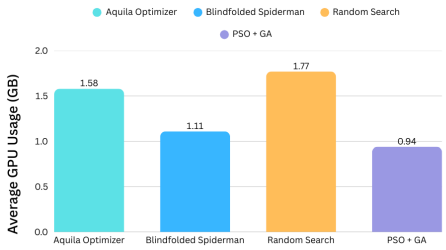

Fig. 3. Average GPU memory usage per evaluation.

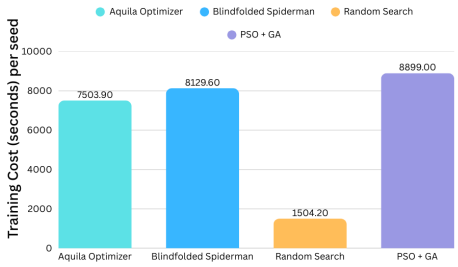

Fig. 4. Average training time per seed.

Across all metrics, BSO consistently provides the strongest overall performance, combining fast convergence, high final accuracy, and stable behaviour across seeds and evaluations. It reaches strong solutions early and maintains a high median performance, indicating that good configurations are found reliably rather than by chance. PSO+GA also performs well but at a substantially higher computational cost, while Aquila converges quickly yet plateaus below the best-performing methods. Random Search is computationally cheap but unreliable, shows highly variable and generally weak outcomes, with sharp GPU usage spikes and poor optimization reliability.

Overall, these results support the conclusion that trajectory-based metaheuristics such as BSO are particularly effective for LoRA hyperparameter optimisation under tight computational constraints.

IV. Multi-objective Extension

In this section, this optimisation problem will be extended from a single-objective optimisation to a bi-objective optimisation setting, where the validation error is minimised while the number of training parameters is also minimised.

A. Bi-objective Formulation

The general multi-objective optimization problem involves multiple optimization goals that conflict with each other. It is represented in this manner:

$$\min F(x) = \big(f_1(x),\, f_2(x),\, \ldots,\, f_M(x)\big)^T$$

This is a bi-objective problem (M = 2), so each candidate (x) of the LoRA configuration is treated as a solution with two optimization goals that conflict with each other:

- **Objective 1**: To minimise the validation error: val_acc(x) is the validation accuracy achieved after fine-tuning the DistilBERT model using the LoRA hyperparameters that were encoded by 'x'.

$$f_1(x) = 1 - \text{val_acc}(x)$$

- **Objective 2:** To minimise the number of training parameters: This function defines the number of trainable parameters (P(x)) in the LoRA adapters, including the classification head.

$$f_2(x) = \mathrm{P}(x)$$

The bi-objective optimisation problem can therefore be generally represented in this manner:

$$\min\big(f_1(x),\, f_2(x)\big)$$

These objectives are conflicting because improving the validation accuracy will require a higher LoRA rank or more LoRA adapters, thereby resulting in an increase in the number of training parameters. Using the validation error instead of accuracy means that for both objectives, smaller values are better, which makes decomposition algorithms more convenient for this research. The goal of multi-objective optimization is not to find one single best point, but a Pareto-optimal solution set, which is a collection of solutions in which the improvement of one objective does not worsen the other. For this research, a bi-objective formulation is important because the main purpose of LoRA is parameter-efficient fine-tuning and this task involves fine-tuning the DistilBERT model using LoRA adapters, and so, the only way to reduce the training parameters is through the LoRA configuration. It also provides a more practical and efficient use of resources during training.

B. Multi-objective Evolutionary Algorithm

The algorithm chosen for the multi-objective extension is called Multi-Objective Evolutionary Algorithm Based on Decomposition with Differential Evolution Operators (MOEA/D-DE). The original MOEA/D decomposes a multi-objective problem into a set of scalar problems using a scalarising function, with each problem associated with a weight vector [5]. This method of decomposition optimizes each problem simultaneously and in a single population. This decomposition technique aligns well with this research because:
(i) The weight vectors represent a distinct trade-off between the validation error and number of trainable parameters.
(ii) Updating each individual using information from the weight vectors of their neighbours helps to smooth out the Pareto front instead of focusing on only the best solutions. To support this, [11] noted that the MOE algorithm based on decomposition is very efficient because the neighbouring subproblems share information, which reduces redundant search.
(iii) [12] also mentioned that the MOE decomposition algorithm is efficient in solving complex multiple optimisation problems.

The MOEA/D-DE has a Differential Evolution (DE) component that provides a stronger mutation that is appropriate for mixed spaces, which is significant for this project. In summary, the MOEA/D-DE algorithm is appropriate for this project because it works very well with minimization problems, and differential evolution works well with the continuous encoding used for LoRA hyperparameters.

C. MOEA/D-DE Configuration and Design

The DistilBERT model with LoRA adapters is fine-tuned on the dair-ai/emotion dataset for 3 epochs per evaluation using the Hugging Face Trainer API. The preprocessing techniques and hyperparameter search space are the same for uniformity. The LoRA configuration is obtained by decoding each solution, 'x', in $[0,1]^6$. The number of trainable parameters, validation accuracy and error, training time, and GPU memory used during training and evaluation were recorded. The parameters used in this experiment include:
I. Population Size: The population size consists of 5 individuals per seed. A small and diverse population allows frequent generations, which makes adaptability possible with limited GPU time.
II. DE Mutation Factor/Scaling Factor (F): The mutation factor (0.5) creates a balance between exploration and stability, as [13] highlights that values between 0.4 and 0.6 promote diversity.
III. Crossover Rate (CR): The crossover rate is (0.9). A higher crossover rate maintains diversity and increases the chances of identifying optimal LoRA configurations.
IV. Neighbourhood size (T): The neighbourhood size was set to 2 neighbours per subproblem. Smaller neighbourhoods are better for balancing information sharing and diversity, and stabilising convergence for costly evaluations.
V. Number of Generations: Three generations were run for each seed apart from the initial generation, therefore producing

20 trials per seed. This works well with the computational constraint, although a larger generation size might perform better.

D. Results and Pareto Front Analysis

In table V, across all seeds, some trade-off solutions that balanced the validation accuracy and number of training parameters were discovered. For each seed, one to two Pareto optimal solutions were returned:
I. Seed 1 Pareto set solution indicates a clean trade-off. It returned two solutions, which show that the trade-off moving from 632k to 668k parameters improves the validation accuracy. **(val_acc = 0.814, params = 632,070), (val_acc = 0.8345, params = 668,934)**
II. Seed 42 returned one dominant solution. This indicates a strong accuracy with moderate parameter count: **(val_acc = 0.893, params = 724,230)**
III. Seed 999 returned a solution with the global best accuracy: **(val_acc = 0.902, params = 853,254)**
IV. Seed 1234 produced two solutions with validation accuracy scores lower than those of the other seeds. This shows that as the number of parameters increases drastically, the accuracy decreases. The solutions returned were: **(val_acc = 0.600, params = 632,070), (val_acc = 0.6495, params = 1,111,302)**
Although the last seed (1234) produced some low accuracy scores, the MOEA/D-DE algorithm returned some high-performance configurations even with the small population size and limited computational resources. This shows that the multi-objective extension is more sufficient than single objective optimisation. Increasing the LoRA rank or alpha values improves accuracy but might inflate the parameter count, which validates the trade-off structure. The different seeds discovered different patterns of the Pareto front, which indicates how valuable multi-seed evaluation is in stochastic MOEAs.
Overall, the Pareto front clearly shows that there is no single best configuration, but a set of different optimal solutions. Table III summarises the performance of the MOEA/D-DE algorithm across the four seeds and all evaluations.

TABLE III
MOEA/D-DE PERFORMANCE ACROSS FOUR SEEDS AND ALL EVALUATIONS.

Measure	**MOEA/D-DE**
Across 4 Seeds	
Best Accuracy (%)	90.20
Mean(%) ± Std(%)	88.26 ± 0.93
Across All Evaluations	
Total Trials/ Trajectories	20
Total Evaluations	80
Mean(%) ± Std(%)	0.6693 ± 0.207
Mean of Seed Bests	0.8826 ± 0.0216
Total Train Time	6400.63 seconds

These results show the effect of LoRA hyperparameters on the trade-off between accuracy and parameter efficiency. This will help to provide guidance when choosing configurations based on resource constraints.

V. CONCLUSION AND FUTURE WORK

This work explored the use of evolutionary and metaheuristic optimization algorithms in fine-tuning the DistilBERT model with LoRA hyperparameters on the emotion classification dataset. The five algorithms implemented in this research are Random search, Aquila, PSO+GA, BSO, and MOEA/D-DE. This project studies how the different algorithms perform in a mixed discrete/continuous hyperparameter space under computational limitations. The single-objective optimization algorithms showed great optimization strength by producing strong validation accuracy scores even with the limited training budget of three epochs. Aquila performed very well, but the Blindfolded Spiderman Optimization (BSO) algorithm surpassed it by achieving the best result with an accuracy score of 94%.

The results show that evolutionary search can improve the efficiency of LoRA despite severe evaluation and computational constraints. The multi-objective extension using MOEA/D-DE aimed to find out how to balance validation accuracy against the number of trainable parameters, while the algorithm did not obtain validation accuracy scores as high as those of the single-objective algorithms, but it achieved its goal of optimizing the trade-offs between the two objectives. It performed quite well, with the best accuracy score being 90.2%, and successfully generated a well-formed Pareto front with non-dominated solutions. The results showed that the algorithm could identify LoRA configurations that were architecturally cheaper while maintaining competitive accuracy. It also clarifies that accuracy alone is not always the best criterion in constrained settings.

Future research could extend this research by working with larger training resources, more trials, and increased computational power, as this research was limited to the GPU run by Google Colab. Increasing the epoch size could change the ranking of the algorithms' performance and reduce stochastic noise. They could also try out various decomposition strategies or hybrid MOEAs since the MOEA/D-DE did not really match the accuracy of the single-objective optimizers. In summary, this study shows that evolutionary optimization is a valuable tool for fine-tuning transformer models using LoRA hyperparameters. Although the single-objective optimization algorithms achieved higher accuracy scores, the multi-objective optimization provided a better understanding of the trade-off between validation accuracy and parameter efficiency.

APPENDIX A
CONTRIBUTION STATEMENT

This appendix summarises the contributions of each group member to the project.

Lohit Ashwa Vaswani

- Experimental Setup & tuning of LoRA Hyperparameters
- Implemented AO and random search.
- Did Experiment analysis
- Report writing

Nneamaka Oluonye

- Implemented a multi-objective evolutionary algorithm
- Report writing

Upasana Das

- Implemented the hybrid PSO-GA algorithm
- Report writing

Shivam Kochhar

- Implemented BSO
- Report writing

A. Json Schema

Listing 1. Generalized Json schema for the storage of algorithms results

```
{
  "method": "<algorithm_name>",
  "gpu_name": "Tesla T4",

  "num_seeds": 4,
  "seeds": [...],

  "algo_metadata": {
    "candidate_size": null, // AO
    "num_trajectories_per_seed": null, // BSO
    "steps_per_trajectory": null, // BSO
    "population_size": null, // PSO+GA
    "total_iterations_per_seed": null // PSO+GA
  },

  "total_trials_per_seed": 20,
  "num_epochs_per_trial": 3,

  "search_space": { ... },

  "seeds_results": [
    {
      "seed": <int>,

      "seed_summary": {
        "best_trial_no": null, // RS, MOEA/D-DE, PSO+GA
        "best_candidate": null, // AO
        "best_trajectory": null, // BSO
        "best_particle": null, // PSO+GA
        "avg_gpu_memory_gb": null
      },

      "best_val_acc": 0.0,
      "val_acc_mean": 0.0,
      "val_acc_std": 0.0,
      "avg_train_time_sec": 0.0,
      "total_train_time_sec": 0.0,

      "pareto_front": [...], // only MOEA/D-DE

      "trials": [
        {
          "trial_no": 1,
          "val_acc": 0.0,
          "train_time_sec": 0.0,
          "gpu_memory_gb": 0.0,
          "hyperparams": { ... },

          "algo_trial_fields": {
            "candidates": [...], // AO
            "particles": [...], // PSO+GA
            "trajectory_no": null, // BSO
            "move_type": null, // BSO
            "generation": null, // MOEA/D-DE
            "subproblem": null, // MOEA/D-DE
            "error": null, // MOEA/D-DE
            "num_params": null // MOEA/D-DE
          }
        }
      ]
    }
  ],

  "overall_stats": {
    "global_best_seed": null,
    "global_best_val_acc": null,
    "global_best_hyperparams": { ... },
    "val_acc_mean_over_all_trials": null,
    "val_acc_std_over_all_trials": null,
    "val_acc_mean_of_seed_bests": null,
    "val_acc_std_of_seed_bests": null,
    "total_train_time_sec_over_all_seeds": null
  }
}
```

TABLE IV
DPSO-GA ALGORITHM HYPERPARAMETERS USED IN OUR EXPERIMENTS.

Parameter	Value
Population size N	5 particles
Number of iterations per seed T	10
Number of random seeds	4 (`1, 42, 999, 1234`)
Epochs per trial	3 fine–tuning epochs
Training subset size	3,000 training examples
DPSO mutation probability ω	0.1
Cognitive crossover probability c_1	0.5
Social crossover probability c_2	0.5
GA crossover probability p_{cx}	0.8
GA mutation probability p_{mut}	0.1
Tournament size (selection)	3

TABLE V
MOEA/D-DEPARETO-OPTIMAL SOLUTIONS ACROSS FOUR SEEDS.

Seed	Best Val. Acc.	Trainable Params	Pareto Front Points
1	0.846 (Trial 3)	1,111,302 (error=0.154)	(0.186, 0.814, 632,070), (0.1655, 0.8345, 668,934)
42	0.893 (Trial 6)	724,230 (error=0.107)	(0.107, 0.893, 724,230)
999	0.902 (Trial 5)	853,254 (error=0.098)	(0.098, 0.902, 853,254)
1234	0.8895 (Trial 1)	724,230 (error=0.1105)	(0.400, 0.600, 632,070),(0.3505, 0.6495, 1,111,302)

B. Population-based heuristic: Aquila Optimizer (AO)

1) Motivation and Justification: The Aquila Optimizer (AO) is a population-based metaheuristic inspired by the hunting strategies of Aquila eagles. Unlike single-point optimizers such as BSO, AO maintains a *population* of candidate solutions and updates them using a combination of global exploration and local exploitation rules. These update equations—four in total—are designed to mimic behavioural transitions observed in eagle hunting: soaring, contour flight,

slow descent, and rapid prey capture. Each behaviour corresponds to a mathematically distinct search operator, giving AO a structured way to adaptively balance diversification and intensification throughout the search.

This property is appealing for LoRA hyperparameter optimisation, which involves navigating a multi-modal and noisy fitness landscape. Each evaluation requires constructing a LoRA-augmented DistilBERT model, fine-tuning it for three epochs, and computing the validation accuracy. Such evaluations are expensive, and the resulting fitness function is both stochastic and non-smooth. AO's population-level sampling allows it to explore multiple hypotheses in parallel, while its exploitation operators focus computational effort on promising regions once discovered. In particular, the mix of Lévy-flight–based exploration, population-mean guidance, and best-solution exploitation is well aligned with the requirement to obtain good configurations under a limited budget.

As with the other methods in this study, AO operates in a continuous space. Each candidate is represented as a vector

$$x \in [0,1]^6.$$

The six components of x are decoded into the actual LoRA configuration using the same `decode_candidate` function used across all algorithms. The components correspond to learning rate (via log scaling), warm-up ratio, LoRA rank, LoRA scaling factor α, dropout probability, and target-module selection (attention-only or attention+FFN). The fitness function $f(x)$ returns the validation accuracy after the standard three-epoch fine-tuning procedure; the objective is to maximise $f(x)$.

2) Population-Based Search Dynamics: AO proceeds in two major phases: *exploration* (early iterations) and *exploitation* (later iterations). The transition is governed by a time factor

$$TF = \frac{t}{T},$$

where t is the current iteration and T is the total number of iterations.

At iteration t, AO maintains:

- the current population $\{x_i\}_{i=1}^{N}$,
- the corresponding fitness values $f(x_i)$,
- personal best values and positions for each candidate,
- a global best $(x_{\text{best}}, f_{\text{best}})$ across all evaluations.

The following operators act on each candidate:

a) Expanded exploration (X1).: Models high-altitude soaring with downward observation. A new point is generated by combining the global best and the population mean:

$$x_i' = x_{\text{best}}(1 - TF) + (x_M - x_{\text{best}} r_2),$$

where $r_2 \sim U(0,1)$. Early in the search ($TF \approx 0$), this produces large exploratory moves; later, the influence of x_{best} increases.

b) Narrowed exploration (X2).: Models contour flight and glide attack using Lévy flights:

$$x_i' = x_{\text{best}} \odot L + x_R + (x_R - x_{\text{best}}) r_2,$$

where L is a Lévy-distributed random vector and x_R is a randomly selected population member. This encourages both large jumps and structured drift around promising regions.

c) Expanded exploitation (X3).: Models low flight with slow descent, guided by the difference between the global best and the population mean:

$$x_i' = x_{\text{best}} \cdot TF - x_M \cdot r_1.$$

As TF increases, the candidate is drawn more strongly toward the best-known region.

d) Narrowed exploitation (X4).: Models the final attack, with highly focused movement and reduced randomness:

$$x_i' = x_{\text{best}} - G_1\, x_i\, r_2 - G_2\, L,$$

where $G_1 = 2(1 - TF)$ and $G_2 = 2TF$. Early iterations use larger G_1 (more correction towards the best), while later iterations reduce the Lévy noise (fine-tuning).

Each update is followed by projection back into $[0,1]^6$, decoding into LoRA hyperparameters, and evaluation of the validation accuracy. AO uses greedy replacement: a candidate is updated only if the new fitness is strictly better. Personal and global bests are updated accordingly. Because each update logs training time, GPU memory usage, and step size, the algorithm also provides detailed resource profiles across the entire run.

3) Pseudocode: Algorithm 3 summarises AO as implemented in this work. The notation follows the conventions used for BSO: `decode` maps a point in $[0,1]^6$ to a LoRA configuration, and `eval` returns validation accuracy after three-epoch fine-tuning.

Algorithm 3 Aquila Optimizer (AO) for LoRA Hyperparameter Optimisation

Require: population size N, iterations T, $\texttt{decode}(\cdot)$, $\texttt{eval}(\cdot)$
Ensure: $x_{\text{best}}, f_{\text{best}}$

Initialise $x_i \sim U([0,1]^6)$ for $i = 1, \ldots, N$
for $i = 1$ **to** N **do**
  $\text{cfg}_i \leftarrow \texttt{decode}(x_i)$
  $f_i \leftarrow \texttt{eval}(\text{cfg}_i)$
end for
$x_{\text{best}} \leftarrow \arg\max_i f_i; \quad f_{\text{best}} \leftarrow \max_i f_i$
Store personal bests $(x_i^{\text{pbest}}, f_i^{\text{pbest}}) \leftarrow (x_i, f_i)$
for $t = 1$ **to** T **do**
  $TF \leftarrow t/T$
  $x_M \leftarrow \frac{1}{N}\sum_i x_i$
  for $i = 1$ **to** N **do**
    Sample $r_1, r_2 \sim U(0,1)$
    if $TF < \frac{2}{3}$ **then**
      {exploration}
      if $r_1 < 0.5$ **then**
        $\{X_1\}$
        $x_i' \leftarrow x_{\text{best}}(1 - TF) + (x_M - x_{\text{best}} r_2)$
      else
        $\{X_2\}$
        Sample Lévy step L
        Select random x_R from population
        $x_i' \leftarrow x_{\text{best}} \odot L + x_R + (x_R - x_{\text{best}}) r_2$
      end if
    else
      {exploitation}
      if $r_1 < 0.5$ **then**
        $\{X_3\}$
        $x_i' \leftarrow x_{\text{best}} \cdot TF - x_M \cdot r_1$
      else
        $\{X_4\}$
        Sample Lévy step L
        $G_1 = 2(1 - TF), \quad G_2 = 2TF$
        $x_i' \leftarrow x_{\text{best}} - G_1 x_i r_2 - G_2 L$
      end if
    end if
    $x_i' \leftarrow \text{clip}(x_i', 0, 1)$
    $\text{cfg} \leftarrow \texttt{decode}(x_i')$
    $f' \leftarrow \texttt{eval}(\text{cfg})$
    if $f' > f_i$ **then**
      $x_i \leftarrow x_i'; \quad f_i \leftarrow f'$
    end if
    if $f' > f_i^{\text{pbest}}$ **then**
      $(x_i^{\text{pbest}}, f_i^{\text{pbest}}) \leftarrow (x_i', f')$
    end if
    if $f' > f_{\text{best}}$ **then**
      $(x_{\text{best}}, f_{\text{best}}) \leftarrow (x_i', f')$
    end if
  end for
end for
return $x_{\text{best}}, f_{\text{best}}$

References

[1] L. Abualigah, D. Yousri, M. Abd Elaziz, A. A. Ewees, M. A. Al-Qaness, and A. H. Gandomi, "Aquila optimizer: a novel meta-heuristic optimization algorithm," *Computers & Industrial Engineering*, vol. 157, p. 107250, 2021.

[2] Z. Han, C. Gao, J. Liu, J. Zhang, and S. Q. Zhang, "Parameter-efficient fine-tuning for large models: A comprehensive survey," *arXiv preprint arXiv:2403.14608*, 2024.

[3] Y.-A. Fan and C.-K. Liang, "Hybrid discrete particle swarm optimization algorithm with genetic operators for target coverage problem in directional wireless sensor networks," *Applied Sciences*, vol. 12, no. 17, p. 8503, 2022.

[4] S. Mittal, "Blindfolded spider-man optimization: A single-point meta-heuristic suitable for continuous and discrete spaces," Master's thesis, California State University, Fresno, 2023.

[5] Y. Xie, Y. Hou, J. Qiao, and B. Yin, "An improved decomposition-based multi-objective evolutionary algorithm with enhanced differential evolution strategy," in *2019 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2019, pp. 2245–2251.

[6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models." *ICLR*, vol. 1, no. 2, p. 3, 2022.

[7] M. Q. Ibrahim, N. K. Hussein, D. Guinovart, and M. Qaraad, "Optimizing convolutional neural networks: A comprehensive review of hyperparameter tuning through metaheuristic algorithms," *Archives of Computational Methods in Engineering*, pp. 1–38, 2025.

[8] L. Abualigah, D. Yousri, M. Abd Elaziz, A. A. Ewees, M. A. Al-qaness, and A. H. Gandomi, "Aquila optimizer: A novel meta-heuristic optimization algorithm," *Computers Industrial Engineering*, vol. 157, p. 107250, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0360835221001546

[9] H. Face, "dair-ai/emotion dataset," https://huggingface.co/datasets/dair-ai/emotion, accessed: 2025-12.

[10] ——, "distilbert-base-uncased," https://huggingface.co/distilbert-base-uncased, accessed: 2025-12.

[11] C.-Y. Zhang and G.-X. Wu, "A decomposition multi-objective evolutionary algorithm based on vector arrangement and strategy adaptation mechanism," in *2023 8th International Conference on Intelligent Computing and Signal Processing (ICSP)*, 2023, pp. 1319–1323.

[12] S. Z. Martinez and C. A. C. Coello, "A multi-objective evolutionary algorithm based on decomposition for constrained multi-objective optimization," in *2014 IEEE Congress on Evolutionary Computation (CEC)*, 2014, pp. 429–436.

[13] A. Angelov and M. Lazarova, "Hybrid artificial bee colony algorithm for test case generation and optimization," *Algorithms*, vol. 18, no. 10, 2025. [Online]. Available: https://www.mdpi.com/1999-4893/18/10/668